

Setup -

1. Anaconda
2. GitBash
3. VS Code
4. AWS account

Project Structure -

```
print("SCRIPT STARTED")

import os
from pathlib import Path

project_name = "us_visa"

list_of_files = [
    f"{project_name}/__init__.py",
    f"{project_name}/components/__init__.py",
    f"{project_name}/components/data_ingestion.py",
    f"{project_name}/components/data_validation.py",
    f"{project_name}/components/data_transformation.py",
    f"{project_name}/components/model_trainer.py",
    f"{project_name}/components/model_evaluation.py",
    f"{project_name}/components/model_pusher.py",
    f"{project_name}/configuration/__init__.py",
    f"{project_name}/constants/__init__.py",
    f"{project_name}/entity/__init__.py",
    f"{project_name}/entity/config_entity.py",
    f"{project_name}/entity/artifact_entity.py",
    f"{project_name}/exception/__init__.py",
    f"{project_name}/logger/__init__.py",
    f"{project_name}/pipeline/__init__.py",
    f"{project_name}/pipeline/training_pipeline.py",
    f"{project_name}/pipeline/prediction_pipeline.py",
    f"{project_name}/utils/__init__.py",
    f"{project_name}/utils/main_utils.py",
    "app.py",
    "requirements.txt",
    "Dockerfile",
    ".dockerignore",
    "demo.py",
    "setup.py",
```

```

        "config/model.yaml",
        "config/schema.yaml",
    ]

for filepath in list_of_files:
    filepath = Path(filepath)
    filedir, filename = os.path.split(filepath)
    if filedir != "":
        os.makedirs(filedir, exist_ok=True)
        print(f"Creating directory: {filedir}")
    if not os.path.exists(filepath) or os.path.getsize(filepath) == 0:
        with open(filepath, "w") as f:
            pass
        print(f"Creating file: {filepath}")
    else:
        print(f"{filepath} already exists")

```

Project Overview -

US Visa Approval Prediction -

1. Understanding the problem statement
2. Understanding the solution
3. Code understanding & Walkthrough
4. Understanding the deployment

Deployment -

1. Docker
2. Cloud services
3. Adding self hosted runner
4. workflows

Problem Statement -

Given certain set of features such as (continent, education, job_experience, training, employment, current_age)

We have to predict whether the application for the visa will be approved or not.

Solution approach -

1. Machine Learning - Classification algorithm
2. Deep Learning - Custom ANN with sigmoid activation function

Solution Proposed -

1. Load the data from DB
2. Perform EDA and feature engineering to select the desirable features
3. Fit the ML Classification Algorithm and find out which one performs better.
4. Select top few and tune hyperparameters
5. Select the best model based on desired metrics

Project Setup -

1. Github Repository
2. Requirements
3. Template

Steps -

1. Create a github repository.
2. Clone the repository to the local folder
3. Open VS code through the Git bash
 - Git clone [repo link]
 - Cd [Repository name] - to enter the repo
 - Code . to open VS code in this folder
4. Create project structure - [template.py](#)
5. Evidently AI - MLOps tool

Git commands -

```
'' 'bash
Git add .
Git commit -m "Updated"
Git push origin main
'''
```

Set up requirements -

```
ipykernel
pandas
numpy
matplotlib
plotly
seaborn
scipy
scikit-learn
imblearn
xgboost
lightgbm
catboost
Pymongo - connection with mongodb
from_root
evidently==0.2.8 - MLOps tool
dill
```

```
PyYAML - managing YAML file
neuro_mf
Boto3 - AWS connection
mypy-boto3-s3
botocore
Fastapi - frontend application
Uvicorn - for Fastapi
Jinja2 - for Fastapi
Python-multipart - for Fastapi
-e . - this is to convert each folder locally into a package
```

[Setup.py](#) -

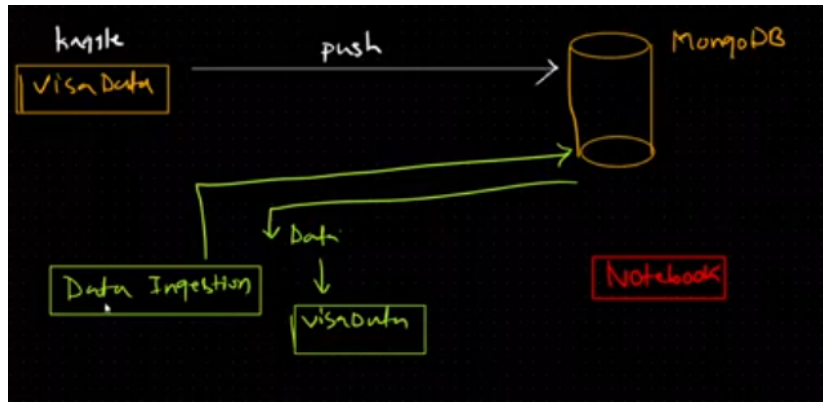
```
setup.py
1 from setuptools import setup, find_packages
2 setup(
3     name="us_visa",
4     version="0.0.0",
5     author="Sree",
6     author_email="ysree2002@gmail.com",
7     packages=find_packages()
8
9 )
```

Database setup & Project Utility

Agenda -

1. Database setup - MongoDB Atlas
2. Logging module
3. Exception module
4. Utility module

– work in jupyter notebook, Once the code looks fine do modular coding.



Create a notebook - mongodb Demo

Now download the dataset and save it notebook folder in the Project.

Import pandas as pd

```
Df = pd.read_csv("Easyvisa.csv")
```

```
df.head()
```

```
df.shape()
```

The dataset is having (25480, 12).

Create Logging Module - to track the activity in the environment.

Create Exception module - to handle errors in the env

Create Utils module - It is a utility module with required customized functions to make it easier for us. Helps in reducing repetitive coding.

After Pushing these changes into Github repo.

Create a Notebook Folder.

- In the Notebook Folder, Create an EDA.ipynb
 - Now, In this notebook perform Exploratory Data Analysis on the dataset.
 - Load the dataset into EDA.ipynb.

```
df = pd.read_csv('EasyVisa.csv')
df.head()
```

-
- Shape of the dataset -

```
df.shape
✓ 0.0s
(25480, 12)
```

-
- Non-null count and Data types of the dataset

```
df.info()
✓ 0.0s

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 25480 entries, 0 to 25479
Data columns (total 12 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   case_id                              25480 non-null  object
1   continent                            25480 non-null  object
2   education_of_employee                25480 non-null  object
3   has_job_experience                   25480 non-null  object
4   requires_job_training                25480 non-null  object
5   no_of_employees                     25480 non-null  int64
6   yr_of_estab                         25480 non-null  int64
7   region_of_employment                25480 non-null  object
8   prevailing_wage                     25480 non-null  float64
9   unit_of_wage                        25480 non-null  object
10  full_time_position                  25480 non-null  object
11  case_status                         25480 non-null  object
dtypes: float64(1), int64(2), object(9)
memory usage: 2.3+ MB
```

Descriptive Stats -

```
df.describe()
✓ 0.0s
```

	no_of_employees	yr_of_estab	prevailing_wage
count	25480.000000	25480.000000	25480.000000
mean	5667.043210	1979.409929	74455.814592
std	22877.928848	42.366929	52815.942327
min	-26.000000	1800.000000	2.136700
25%	1022.000000	1976.000000	34015.480000
50%	2109.000000	1997.000000	70308.210000
75%	3504.000000	2005.000000	107735.512500
max	602069.000000	2016.000000	319210.270000

Python

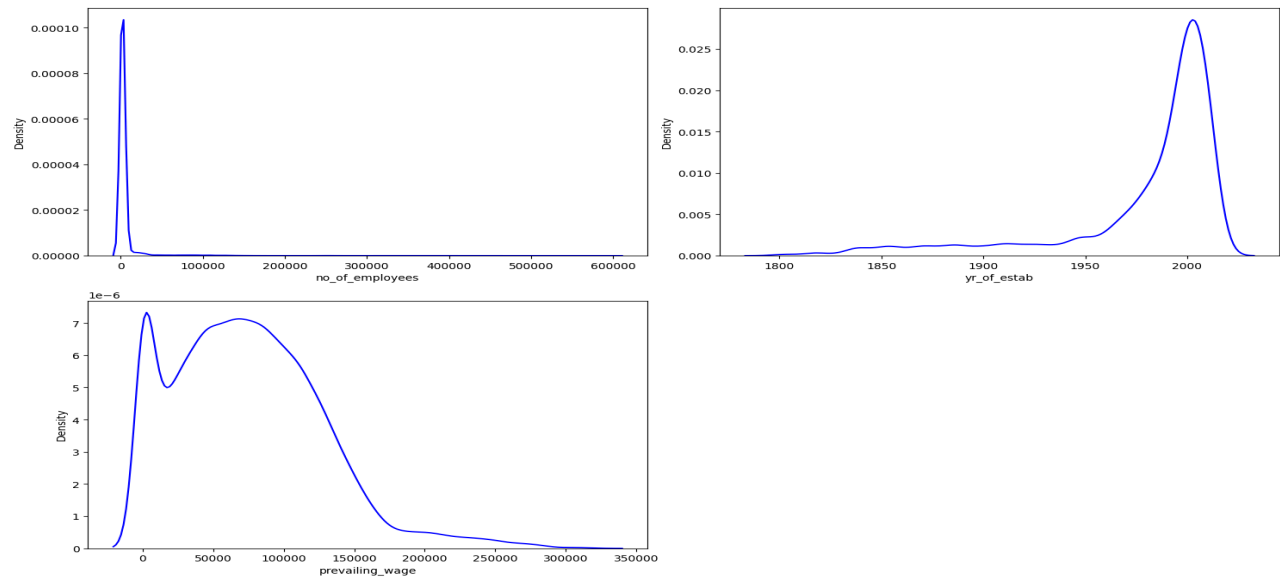
Exploratory Data Analysis -

Numerical and Categorical Features -

Numerical features: ['no_of_employees', 'yr_of_estab', 'prevailing_wage']
Categorical features: ['case_id', 'continent', 'education_of_employee', 'has_job_experience', 'requires_job_training', 'region_of_employment', 'unit_of_wage', 'full_time_position', 'case_status']

UNIVARIATE ANALYSIS of Numerical Features -

Univariate Analysis of Numerical Features



- `no_of_employees` column looks more skewed but it may be because of outliers.
- `yr_of_estab` is a discrete feature.
- `prevailing_wage` is a right-skewed distribution.

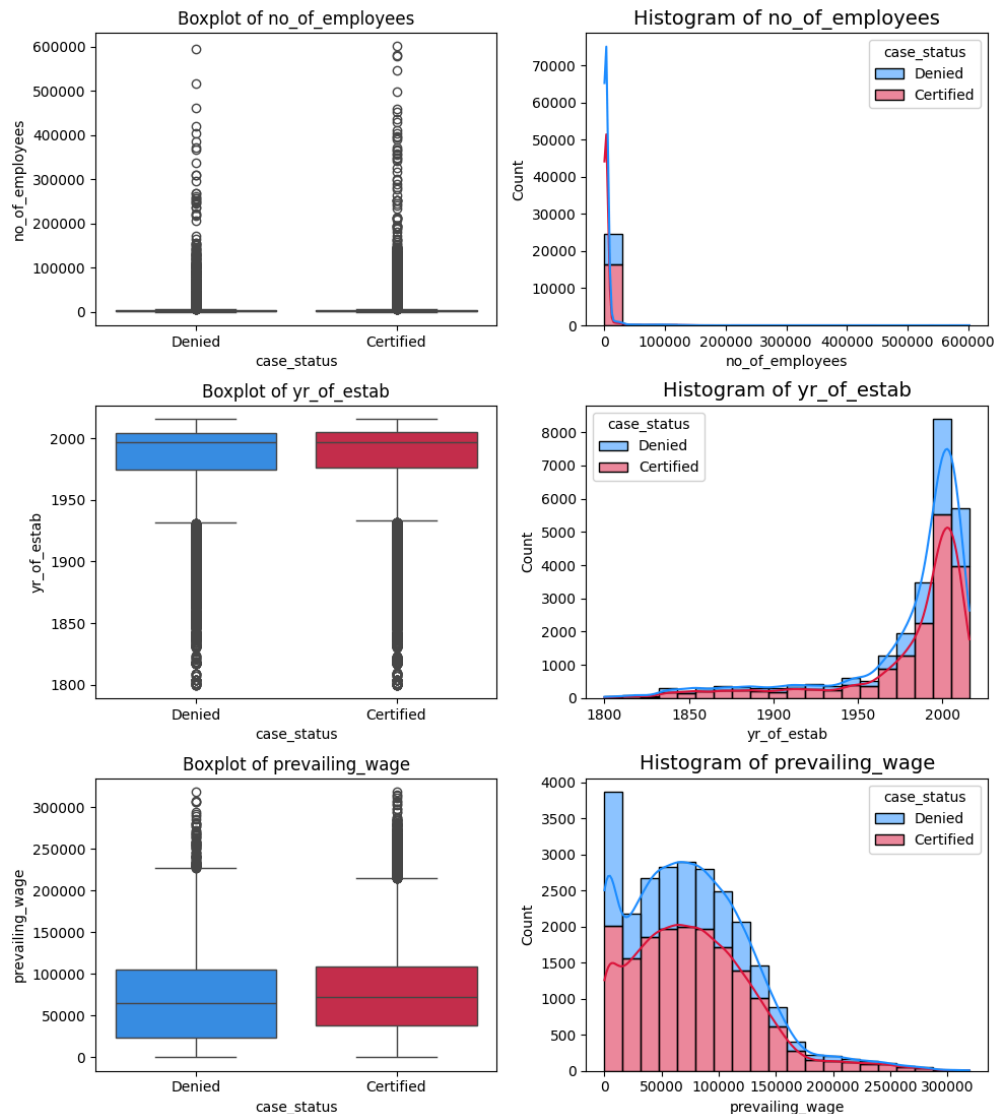
MULTIVARIATE ANALYSIS -

Categorical Columns -

Performed Chi - squared test on the categorical features. From the results, we can determine that column `requires_job_training` is not a significant categorical column.

Numerical Columns -

Distribution of Numerical Features By Case Status



Overall Insights -

- No of Employees has many outliers which can be Handled in Feature Engineering and no_of_employees is Right Skewed.
- yr_of_estab is left skewed and some outliers below the lower bound of Box plot.
- prevailing_wage is right skewed with outliers above upper bound of box plot.
- There are No missing values in the dataset.

The case_id column can be deleted because each row has unique values.

- The case_status column is the target to predict.
- In the Categorical column, features can be made Binary numerical in feature Encoding

After completing EDA -
Started Modular coding

Created constants, configuration.

Now I have to establish a connection between the MongoDB client to work on it in the project.

Hence, I created a class - with the help of constants (connection url, database_name, collection_name).

Workflow -

1. Constants
2. Entity
3. Components
4. Pipelines
5. Main file

Data Drift with Evidently and Data Validation -

- Data Drift demo with Evidently
- Data Validation - Validate your data. In my dataset I have features. Before training data checking if the data is in correct format.

Data Drift - When distribution of test dataset is not similar to distribution of training dataset. It is a problem of data drift.

To solve this problem manually -

Python - we use seaborn and matplotlib libraries and plot train data and test data distributions. Check them. Handle them and again look at distributions.

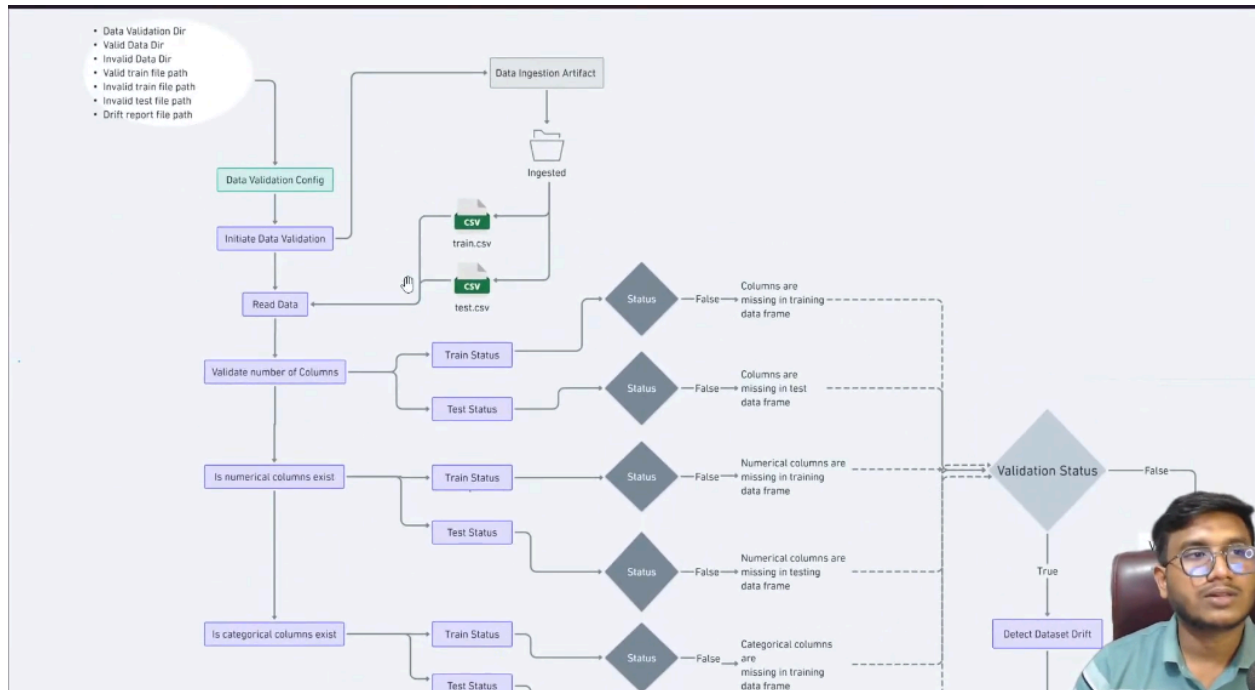
Solving data drift - REsample the data.

How can we detect data drift -

- Create notebook - data_drift_demo_evidently.ipynb
- Import necessary libraries
- From evidently.dashboard import Dashboard
- From evidently.tabs import DataDriftTab, CatTargetDriftTab
- From evidently.model_profile import Profile
- From evidently.profile_selections import

- Load dataset using pandas
- #Data Drift Dashboard
- Train Test Split
- Create a dashboard
- Before passing data to the dashboard. You need to create tabs
- Example - `boston_datadrift_dashboard = Dashboard(tabs=[DataDriftTab()])`

DATA VALIDATION -



1. Validate number of columns - check if all columns are there or not.
2. Check if numerical columns exist.
3. Check if Categorical columns exist.
4. If the above three points are validated then check for data drift.

Data Transformation -

Feature Engineering -

- Handling missing values
- Handling duplicates
- Check data types
- Understand the dataset
- Converted company year to age
- Train test split
- Feature transformation - because data is not distributed properly.
- To balance the imbalance classes we use SMOTENN library.

5. Modular coding - Data Ingestion
6. Data Validation → verify schema of data, data drift detection
7. Data Transformation
8. Model trainer → hyperparameter tuning (neuromf)
9. Model evaluation
10. Model pusher to production (Amazon S3)
11. Prediction pipeline
12. User app (front end) → Fast API

Model Evaluation -

Calculate Accuracy score, F1, Precision, Recall

Threshold score = 0.7

Only if the model passes the threshold score the trained model is pushed into S3 bucket.

We will have

1. Trained model
2. Production model

Test data is tested on both trained model and production model.

Model Pusher -

AWS S3 bucket. When the accuracy of the trained model is more than production model. We will execute model pusher and the training model becomes the production model.

AWS CI/CD Deployment -

In AWS,

1. Create S3 bucket
2. Go to IAM
3. Create access keys- access key and secret access key
4. Give access to S3 service
5. Save keys as environment variables
6. Create aws_connection.py to establish a connection with AWS - has S3client class in it.
7. Create a cloud storage folder in us_visa.
8. Then a constructor file → __init__.py
9. Create aws_storage.py file in that folder. All the operations that happen in AWS code happen here.
10. Functionality in S3 bucket using python code. Following functions -
 - a. Load_object
 - b. Upload_file
 - c. Load_model
11. Create a file in entity and name it s3_estimator.py

Now, go to model_evaluation.py

CI/CD Deployment - Continuous Integration / Continuous Delivery

Upload Github → AWS

1. Github action → setup → automated (All you have to do is a yaml file)
2. Jenkins → manual setup
3. Circle CI → manual setup

Tools -

- Amazon EC2 → virtual machine
- Amazon ECR → Docker image
- Github Action → self hosted runner

AWS - CI/CD - Deployment-with-Github-Actions

1. Login to AWS console
2. Create IAM user for deployment
 - i. EC2 access: It is a virtual machine
 - ii. ECR: Elastic container registry to save docker image in AWS

Description: About the deployment

1. Build docker image of the source code.
2. Push your docker image to ECR
3. Launch your EC2
4. Pull your image from ECR in EC2
5. Launch you docker image in EC2

Policy:

1. AmazonEC2ContainerRegistryFullAccess
 2. AmazonEC2FullAccess
3. Create ECR repo to store/save docker image
 4. Create EC2 machine Ubuntu
 5. Configure EC2 as a self-hosted runner
 6. Setup github secrets
 - i. AWS_ACCESS_KEY_ID
 - ii. AWS_SECRET_ACCESS_KEY
 - iii. AWS_DEFAULT_REGION
 - iv. ECR_REPO

Without Docker:

- "It works on my laptop"
- Fails on server
- Version conflicts
- Missing packages

With Docker:

- Same environment everywhere
- Reproducible deployments
- Easy cloud deployment

- CI/CD friendly

In AWS account -

- Login to AWS.
- Create an instance in EC2 machine (Ubuntu)
- Open EC2 and install docker in EC2 machine

Ubuntu commands

```
sudo apt-get update -y
sudo apt-get upgrade
```

```
curl -fsSL https://get.docker.com -o get-docker.sh
sudo sh get-docker.sh
sudo usermod -aG docker ubuntu
newgrp docker
```

```
docker -- version
```

- In Github settings → Developer settings → Personal access tokens → token (classic) → Enable workflows.
- Configure EC2 as a self-hosted runner
 - In Github, go to settings → actions → Runners → New self-hosted runner → choose your os (for Ubuntu - Linux) and copy the Download code
- Setup Github secrets
 - Github settings → Secrets and variables (security) → Actions → repository secrets → new repository secrets → Enter Name - AWS Access key ID and Secret - Secret AWS access key, Add region
 - Before adding ECR repo
 - Create a .github → another folder workflows folder in VS code application → create a file in this folder called aws.yaml (Write all the yaml commands in this file.
- In the docker file - write docker commands

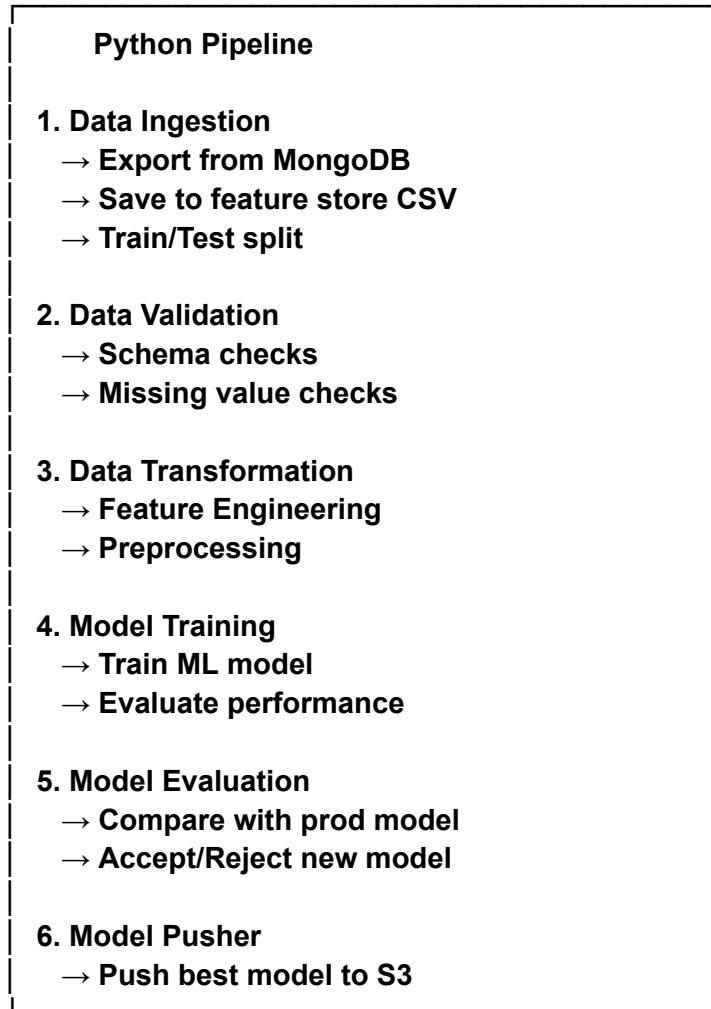
```
FROM python:3.8.5-slim-buster
WORKDIR /app
COPY . /app
RUN pip install -r requirements.txt
CMD ["python3", "app.py"]
```

End to End workflow -

MongoDB (Raw Data)



PyMongo pulls data



Model saved to AWS S3



**Evidently monitors
data drift in production**

Deployment Workflow -

Developer writes code (VS Code)



Git push to main branch



GitHub Actions triggered (aws.yaml)



CI Job (GitHub Runner)



Checkout latest code



Authenticate with AWS



Login to Amazon ECR



Docker build image from Dockerfile



Docker push image → AWS ECR

CD Job (EC2 Self-hosted Runner)



Authenticate with AWS



Login to ECR from EC2



Pull latest Docker image from ECR

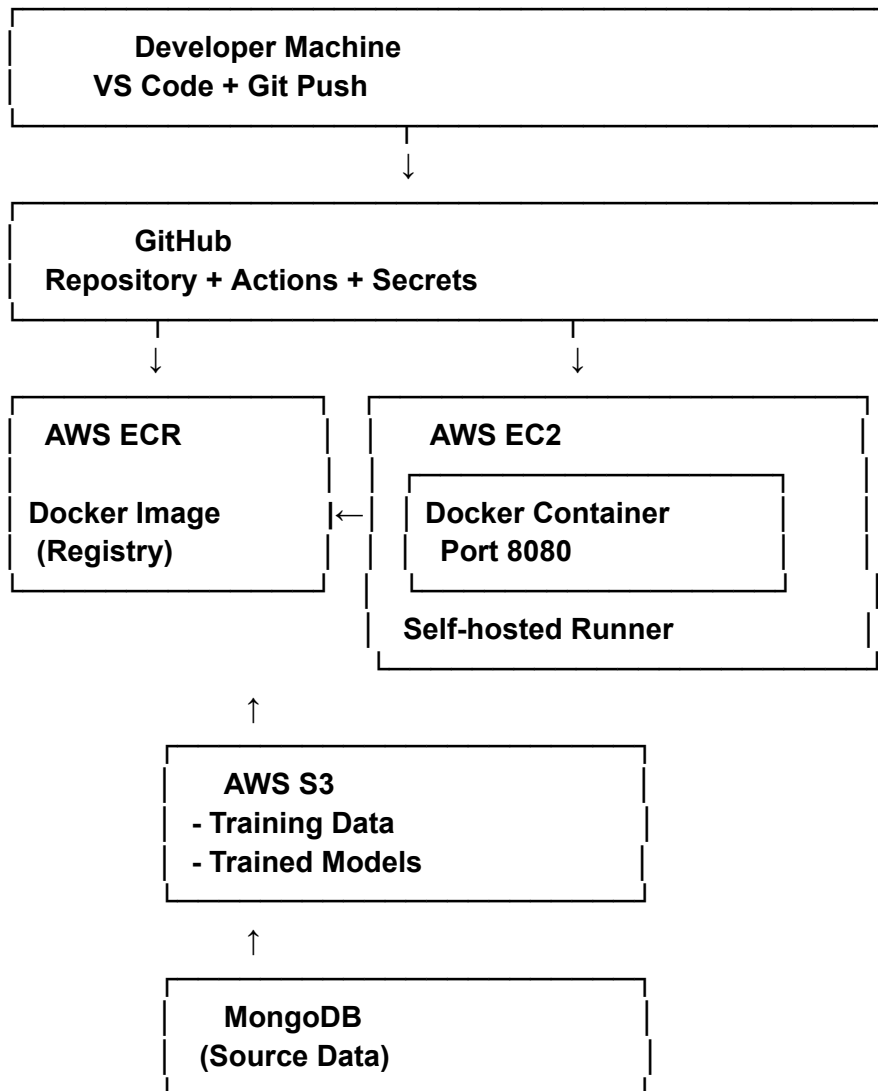


Run container on port 8080



 App Live on EC2 (port 8080)

Deployment Architecture -



Circle CI, Jenkins

